Отчёт по лабораторной работе № 5. Построение графиков

Простейший вариант

Дагделен Зейнап Реджеповна

В отчёте описано освоение синтаксиса языка Julia для построения графиков с использованием пакета Plots.jl и его бэкендов (gr, pyplot, plotlyjs, unicodeplots). Приведены примеры построения графиков функций, точечных графиков, аппроксимации, линий уровня, анимации, а также выполнения самостоятельных заданий.

Содержание

1 Введение **7**
- 1.1 Актуальность . . . 7
- 1.2 Цель работы . . . 7
- 1.3 Задачи . . . 7
- 1.4 Объект и предмет исследования . . . 8
- 1.5 Методы исследования . . . 8

2 Теоретическая часть **9**

3 Материалы и методы **10**

4 Выполнение лабораторной работы **11**
- 4.1 5.2.1. Основные пакеты и бэкенды . . . 11
- 4.2 5.2.2. Опции при построении графика (sin(x) и ряд Тейлора) . . . 15
- 4.3 5.2.3. Точечные графики . . . 16
- 4.4 5.2.4. Аппроксимация данных . . . 19
- 4.5 5.2.5. Две оси ординат . . . 20
- 4.6 5.2.7. Параметрические графики . . . 20
- 4.7 5.2.8. График поверхности . . . 22
- 4.8 5.2.9. Линии уровня . . . 23
- 4.9 5.2.11. Анимация (gif) . . . 24
- 4.10 5.2.12. Errorbars . . . 25
- 4.11 5.2.13. Гистограммы (Distributions) . . . 26
- 4.12 5.2.14. Подграфики . . . 28

5 Самостоятельные задания (5.4) **31**
- 5.1 Задание 1. Все типы графиков sin(x) в одном окне . . . 31
- 5.2 Задание 2. sin(x) со всеми возможными опциями оформления . . . 32
- 5.3 Задание 3. График $y = \pi x^2 \ln(x)$ . . . 32
- 5.4 Задание 4. Вектор $x = (-2, -1, 0, 1, 2)$, $y = x^3 - 3x$ в 4 панелях . . . 33
- 5.5 Задание 5. Два графика $y_1 = \pi x$, $y_2 = e^x \cos(x)$ . . . 34

6 Результаты **36**

7 Обсуждение результатов **37**

8 Заключение **38**

9 Список литературы **39**

Список иллюстраций

4.1 Установка пакетов . . . 12
4.2 График через gr() . . . 13
4.3 График через pyplot() . . . 14
4.4 График через plotly . . . 14
4.5 sin(x) и ряд Тейлора . . . 15
4.6 Сохранённый taylor.png . . . 16
4.7 Точечный график . . . 17
4.8 Точки с разным размером маркера . . . 18
4.9 3D точечный график . . . 18
4.10 Исходные данные . . . 19
4.11 Аппроксимация полиномом 5-й степени . . . 19
4.12 Две оси ординат . . . 20
4.13 Параметрическая кривая на плоскости . . . 21
4.14 Пространственная кривая . . . 21
4.15 Поверхность и каркас . . . 22
4.16 Поверхность g(x,y) . . . 23
4.17 Линии уровня . . . 24
4.18 Анимация поверхности . . . 25
4.19 Errorbars . . . 26
4.20 Гистограмма случайных возрастов . . . 27
4.21 Гистограмма нормального распределения . . . 27
4.22 Смесь распределений . . . 28
4.23 Подграфики в столбик . . . 29
4.24 Подграфики сеткой 2x2 . . . 29
4.25 Объединение разных типов графиков . . . 30

5.1 Все типы графиков sin(x) . . . 31
5.2 sin(x) со всеми опциями . . . 32
5.3 График y = pi x^2 ln(x) . . . 33
5.4 Задание 4 (4 панели) . . . 34
5.5 figure_Dagdelen.png . . . 34
5.6 Задание 5, вариант 1 . . . 35
5.7 Задание 5, вариант 2 . . . 35

Список таблиц

1 Введение

1.1 Актуальность

Визуализация данных — неотъемлемая часть научных и инженерных исследований. Julia предоставляет мощный и гибкий инструмент `Plots.jl`, позволяющий строить графики различных типов с минимальными затратами кода. Освоение этого инструмента важно для дальнейшей работы с данными и подготовки отчётов.

1.2 Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

1.3 Задачи

1. Изучить основные пакеты для работы с графиками в Julia.
2. Освоить построение графиков функций, точечных графиков, аппроксимации, линий уровня, анимации.
3. Выполнить самостоятельные задания (п. 5.4 методических указаний).

1.4 Объект и предмет исследования

Объект — графические возможности языка Julia. Предмет — синтаксис и опции пакета `Plots.jl` и его бэкендов.

1.5 Методы исследования

Экспериментальный метод: выполнение кода в Jupyter Lab / Pluto, фиксация результатов скриншотами, анализ полученных графиков.

2 Теоретическая часть

Julia поддерживает несколько пакетов для построения графиков. Стандартным является `Plots.jl`, который позволяет легко менять бэкенды: `gr()`, `pyplot()`, `plotlyjs()`, `unicodeplots()` [1; 3]. Для статистических расчётов и распределений используются пакеты `Statistics` и `Distributions` [2; 4].

Для построения графиков необходимо задать массивы значений `x` и `y`, а затем вызвать функцию `plot` с нужными опциями (заголовок, подписи осей, цвет, легенда и т.д.) [1].

3 Материалы и методы

- Оборудование: персональный компьютер.
- Программное обеспечение: Julia, Jupyter Lab (или Pluto.jl), пакеты `Plots`, `Statistics`, `Distributions`, `PyPlot`, `PlotlyJS`, `UnicodePlots`, `FreeType`, `FileIO`.
- Последовательность действий: установка пакетов, выполнение примеров из раздела 5.2, выполнение самостоятельных заданий, сохранение скриншотов в папку `image`.

4 Выполнение лабораторной работы

4.1 5.2.1. Основные пакеты и бэкенды

Что делаем и зачем. Устанавливаем и подключаем пакеты, необходимые для графики (`Plots` и его бэкенды), статистики (`Statistics`, `Distributions`) и работы с файлами (`FileIO`). Это подготовительный этап: без `using` пакеты недоступны.

```
using Pkg
Pkg.add(["Plots", "Statistics", "Distributions"])
Pkg.add("PyPlot")
Pkg.add("PlotlyJS")
Pkg.add("UnicodePlots")
using Pkg
Pkg.add(["FreeType", "FileIO"])
using Plots, Statistics, Distributions
```

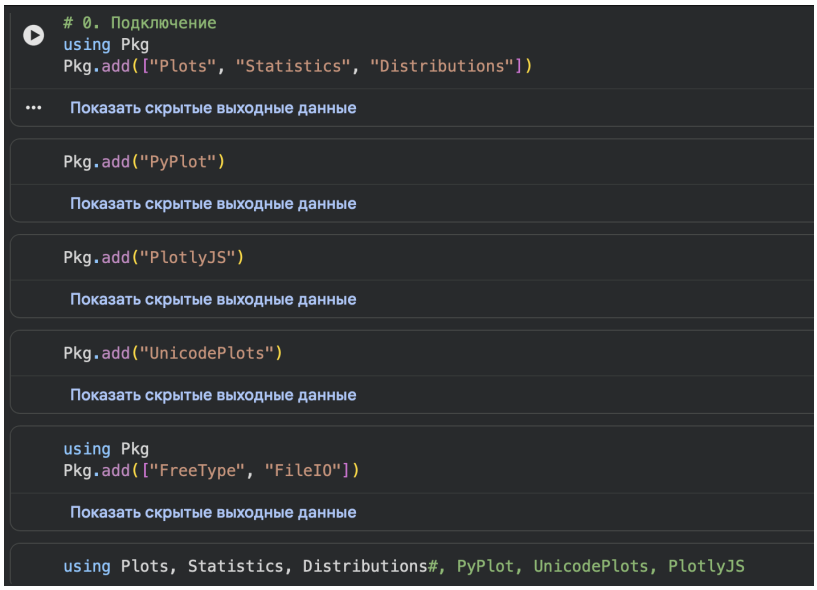


Рисунок 4.1: Установка пакетов

Что делает код. Задаём функцию $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ поэлементно (точки перед операциями — `.^`, `.*`, `exp.`), генерируем массив `x` от −5 до 10 (151 точка) и считаем `y = f(x)`. Затем через бэкенд `gr()` строим график с заголовком, подписями осей, синей линией и легендой.

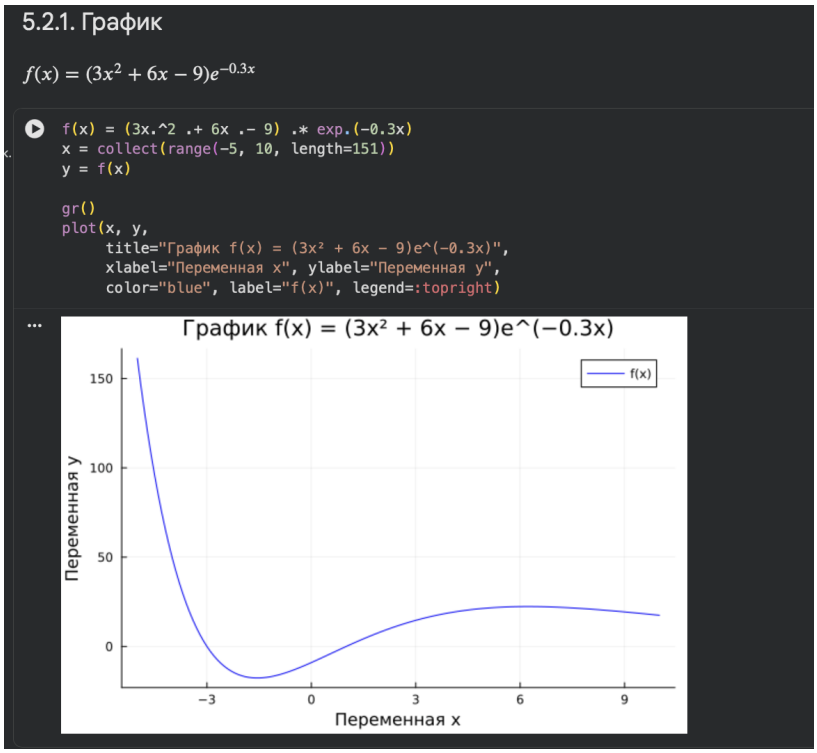


Рисунок 4.2: График через gr()

Зачем менять бэкенд. Один и тот же вызов `plot` с `pyplot()` даёт другой рендер (ближе к matplotlib), а с `plotly` — интерактивный график. Это демонстрирует гибкость `Plots.jl`.

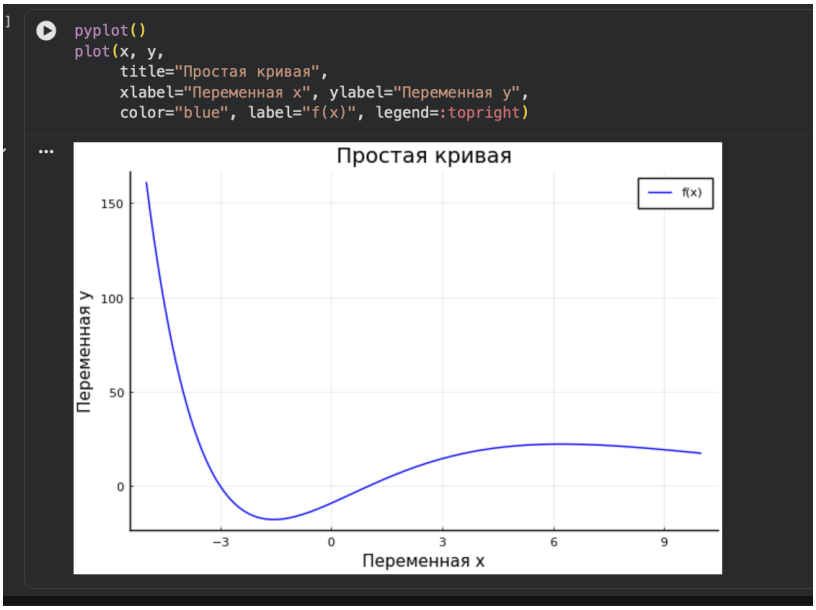


Рисунок 4.3: График через pyplot()

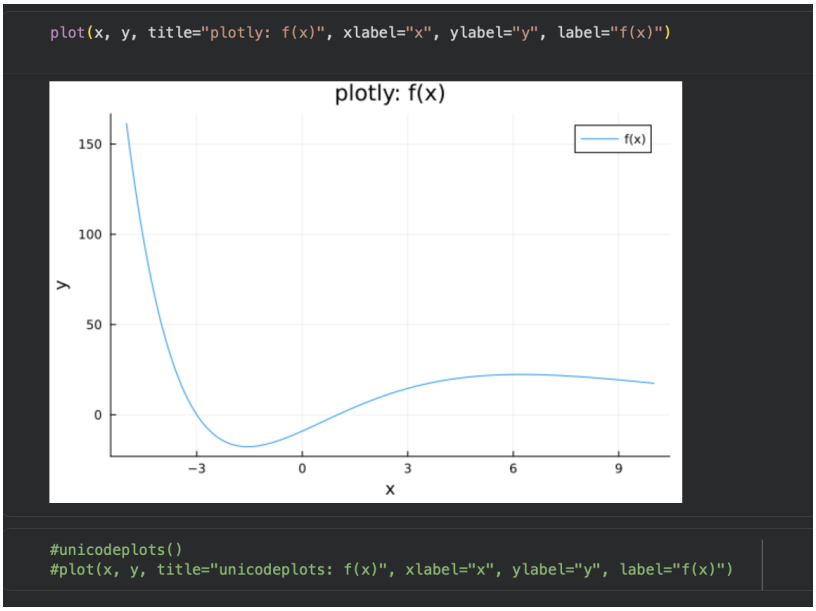


Рисунок 4.4: График через plotly

4.2 5.2.2. Опции при построении графика (sin(x) и ряд Тейлора)

Что делаем. Сравниваем теоретический $\sin(x)$ и его разложение в ряд Тейлора (до 4-й степени) на $[-2\pi, 2\pi]$. Строим две кривые на одном рисунке: чёрный пунктир — теория, синяя линия с заливкой — ряд Тейлора. Показываем, что приближение хорошо работает вблизи нуля и расходится на краях.

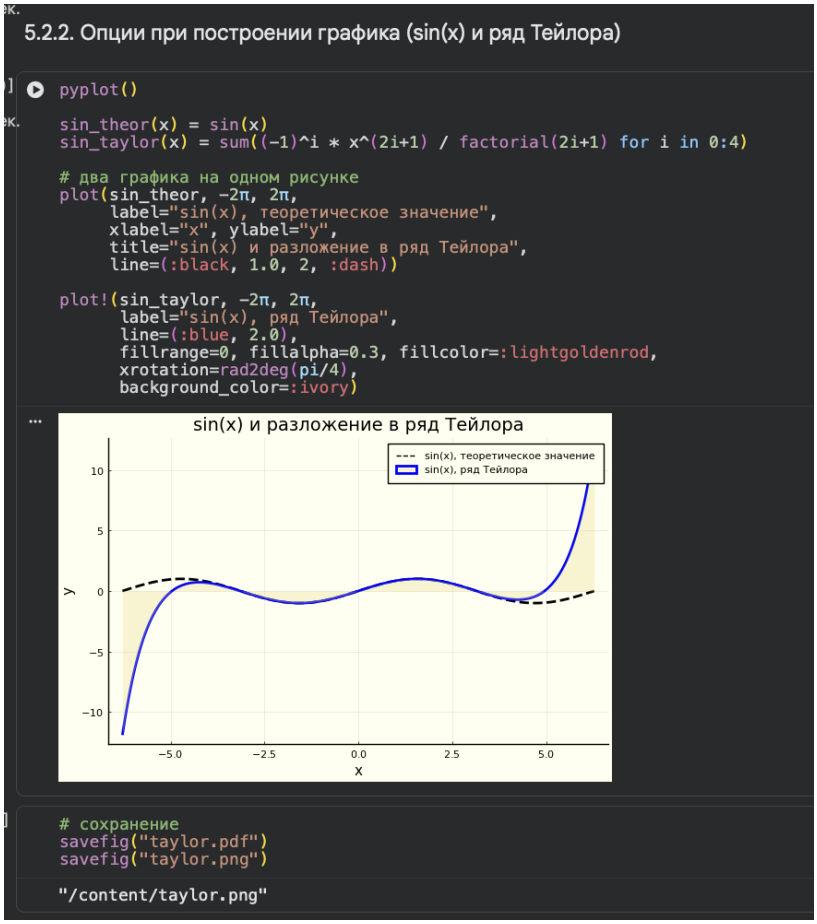


Рисунок 4.5: sin(x) и ряд Тейлора

Сохранение. Функция `savefig` сохраняет текущий график в `taylor.pdf` и `taylor.png` — полезно для вставки в отчёты.

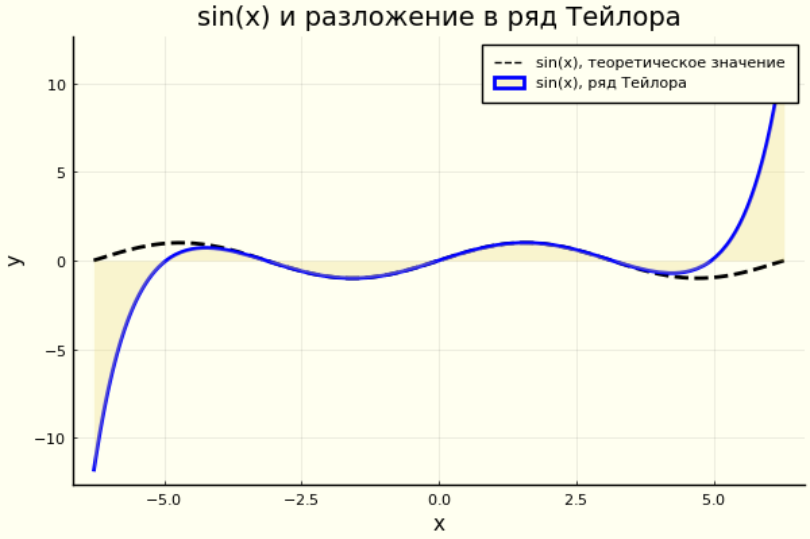


Рисунок 4.6: Сохранённый taylor.png

4.3 5.2.3. Точечные графики

Простой точечный график. 10 случайных значений `y` на отрезке `x` от 1 до 10. Функция `scatter` рисует точки; заданы заголовок, подписи осей и легенда.

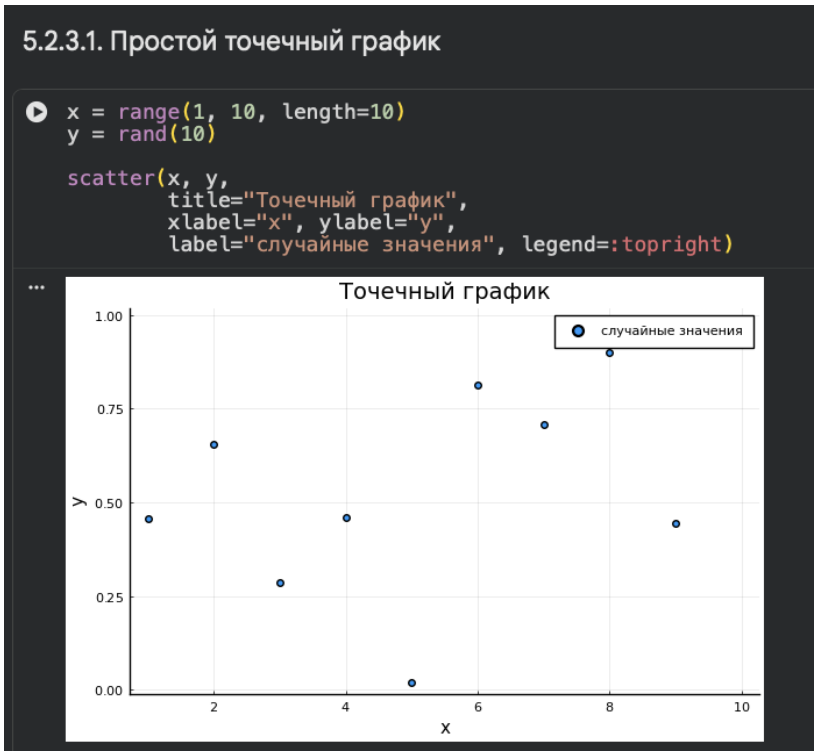


Рисунок 4.7: Точечный график

Кодирование размера. 50 точек, размер маркера задаётся случайно (`markersize=ms`) — так можно визуально кодировать третью переменную.

5.2.3.2. Точечный график с кодированием размера точк

```
n = 50
x = rand(n); y = rand(n); ms = rand(n) .* 30

scatter(x, y, markersize=ms,
        title="Точки с разным размером маркера",
        xlabel="x", ylabel="y",
        label="точки", legend=:topright)
```

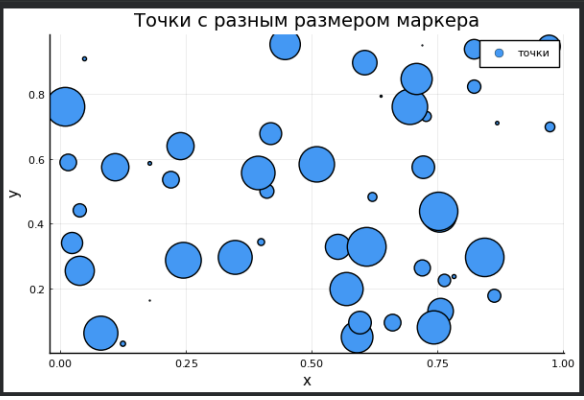


Рисунок 4.8: Точки с разным размером маркера

3D-точечный график. Те же идеи в пространстве (x, y, z); размер маркера снова случайный.

5.2.3.3. 3-мерный точечный график

```
n = 50
x = rand(n); y = rand(n); z = rand(n); ms = rand(n) .* 30

scatter(x, y, z, markersize=ms,
        title="3D точечный график",
        xlabel="x", ylabel="y", zlabel="z",
        label="точки", legend=:topright)
```

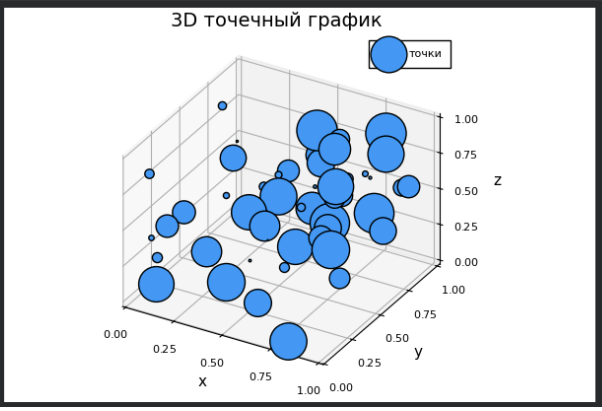


Рисунок 4.9: 3D точечный график

4.4 5.2.4. Аппроксимация данных

Что делаем. Генерируем «экспоненциальные» данные с шумом и аппроксимируем их полиномом 5-й степени методом наименьших квадратов. Матрица Вандермонда `A = [ones x x^2 ... x^5]`, решение `c = A \ y` даёт коэффициенты; затем строим аппроксимирующую кривую поверх облака точек.



Рисунок 4.10: Исходные данные

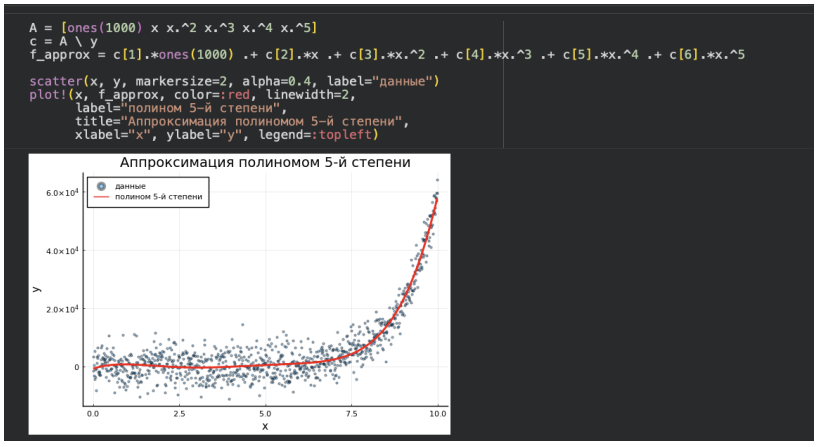


Рисунок 4.11: Аппроксимация полиномом 5-й степени

4.5 5.2.5. Две оси ординат

Зачем. Если две траектории имеют сильно разный масштаб, на одной оси вторая становится нечитаемой. `twinx()` создаёт вторую ось Y. Синяя линия — случайные числа (левая ось), красная — они же ×100 (правая ось).



Рисунок 4.12: Две оси ординат

4.6 5.2.7. Параметрические графики

На плоскости. $x = \sin(t)$, $y = \sin(2t)$, $t \in [0, 2\pi]$ — получается фигура Лиссажу (замкнутая кривая).

5.2.7.1. Параметрический график кривой на плоскости

```
using Plots
gr()

Plots.GRBackend()

t = range(0, 2π, length=500)
x = sin.(t); y = sin.(2t)

plot(x, y, linecolor=:orange,
     title="Параметрическая кривая: x=sin(t), y=sin(2t)",
     xlabel="x", ylabel="y", label="кривая")
```

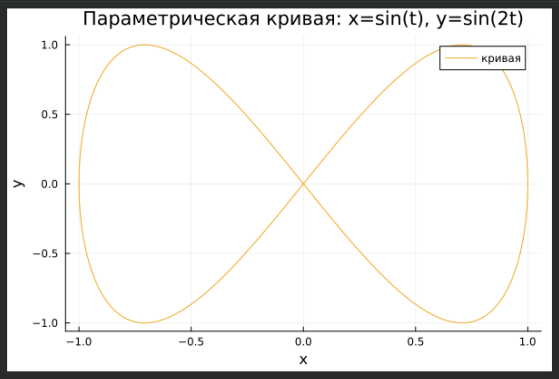


Рисунок 4.13: Параметрическая кривая на плоскости

В пространстве. $x = \cos(t), y = \sin(t), z = \sin(5t), t \in [0, 10]$ — спиралевидная пространственная кривая.

5.2.7.2. Параметрический график кривой в пространстве

```
t = range(0, stop=10, length=1000)
x = cos.(t); y = sin.(t); z = sin.(5t)

plot(x, y, z,
     title="Пространственная кривая: x=cos(t), y=sin(t), z=sin(5t)",
     xlabel="x", ylabel="y", zlabel="z", label="кривая")
```

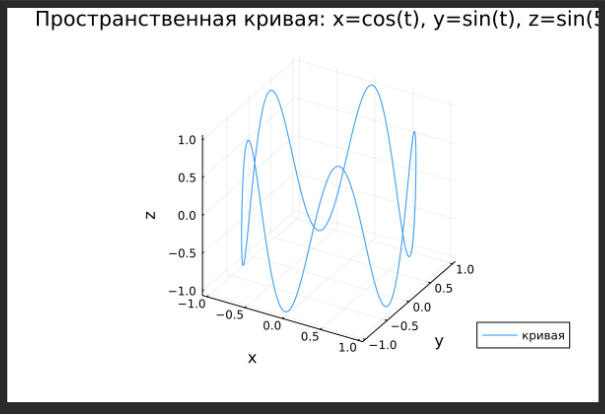


Рисунок 4.14: Пространственная кривая

4.7 5.2.8. График поверхности

Что делаем. Для $f(x, y) = x^2 + y^2$ (параболоид) строим 3D-поверхность с цветовой шкалой (`surface`) и каркасный вариант (`linetype=:wireframe`).

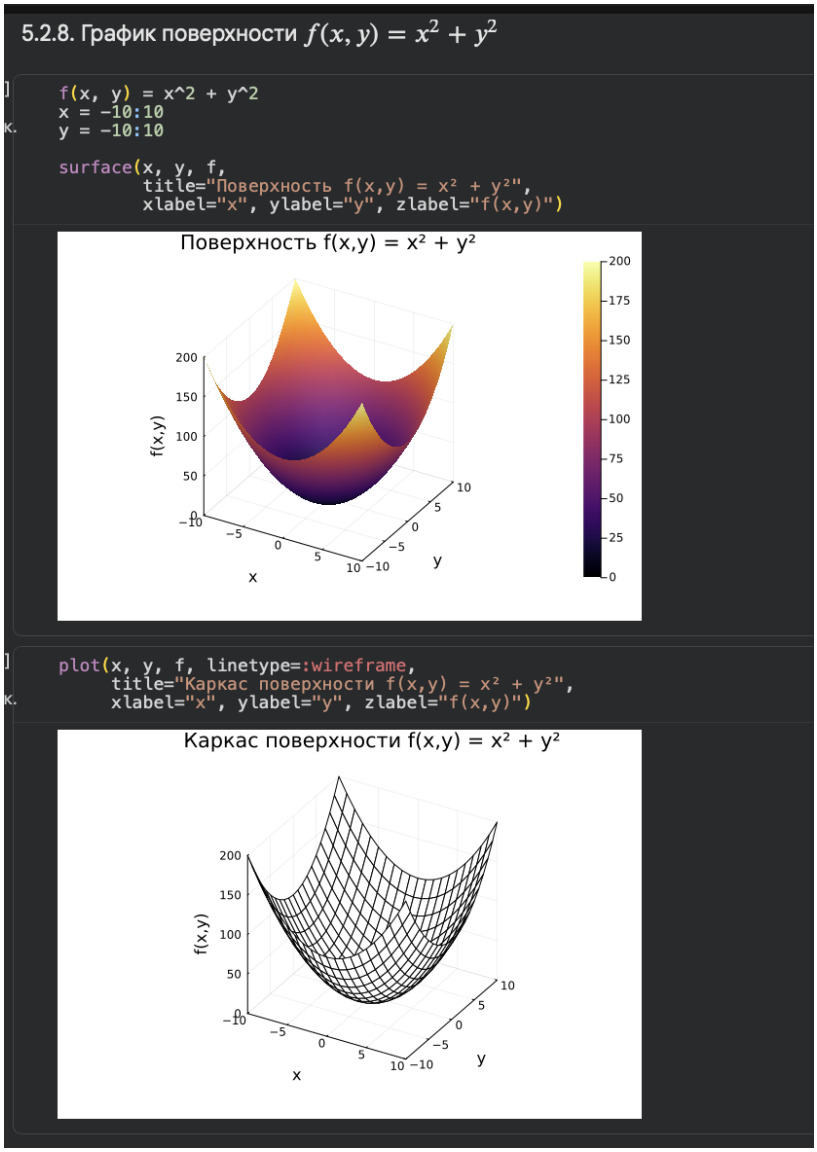


Рисунок 4.15: Поверхность и каркас

4.8 5.2.9. Линии уровня

Что делаем. Для $g(x, y) = (3x + y^2)|\sin(x) + \cos(y)|$ сначала строим 3D-поверхность, затем линии уровня (`contour`) и линии уровня с заливкой (`fill=true`). Линии уровня помогают увидеть «холмы» и «впадины» функции на плоскости.

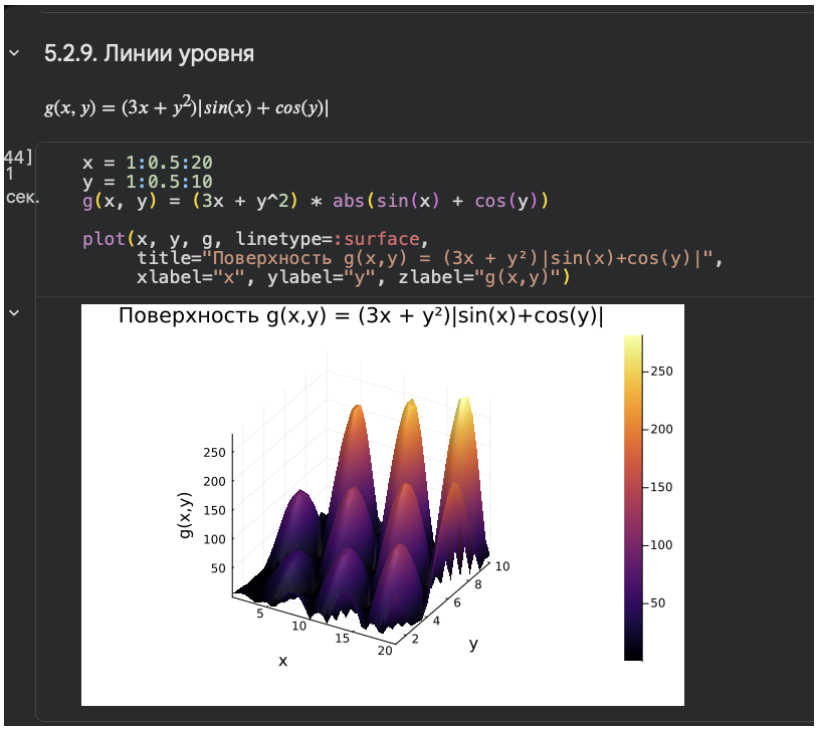


Рисунок 4.16: Поверхность g(x,y)

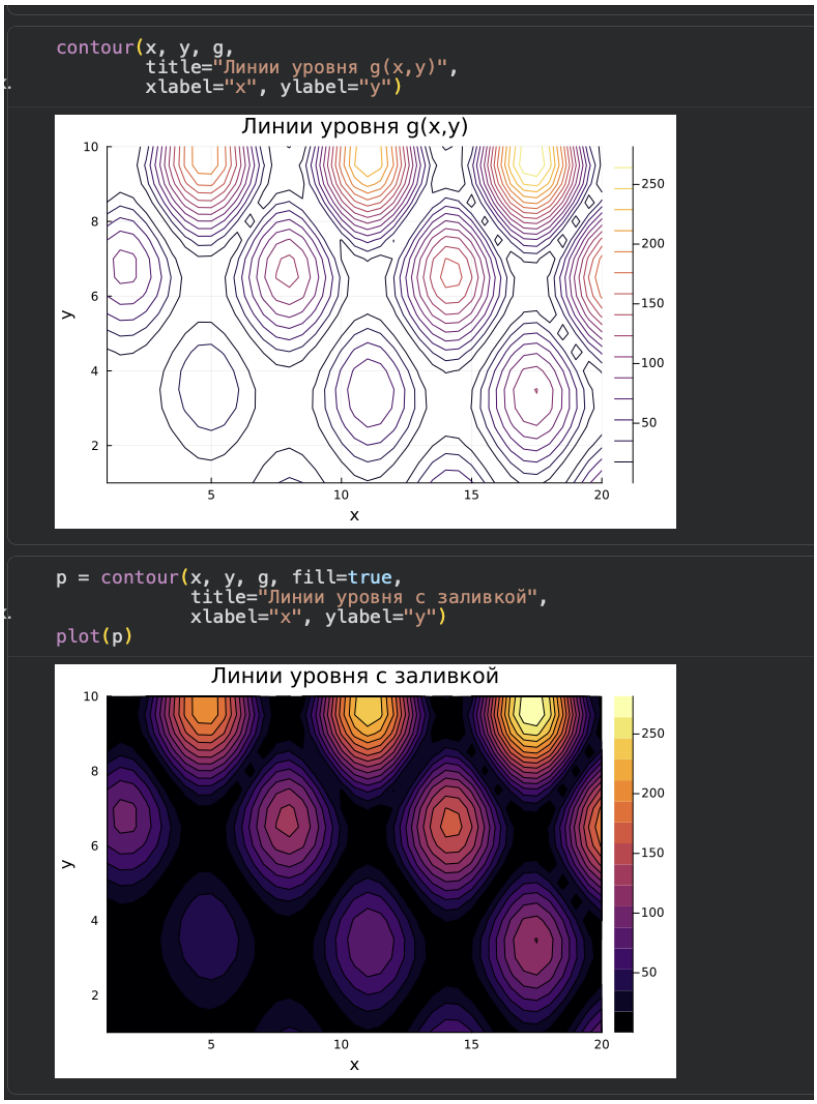


Рисунок 4.17: Линии уровня

4.9 5.2.11. Анимация (gif)

Что делаем. Строим поверхность $z = \sin(x + 10\sin(i)) + \cos(y)$, где i пробегает от 0 до 2π за 100 кадров, и сохраняем в GIF через `@gif`. На скриншоте — два кадра анимации.

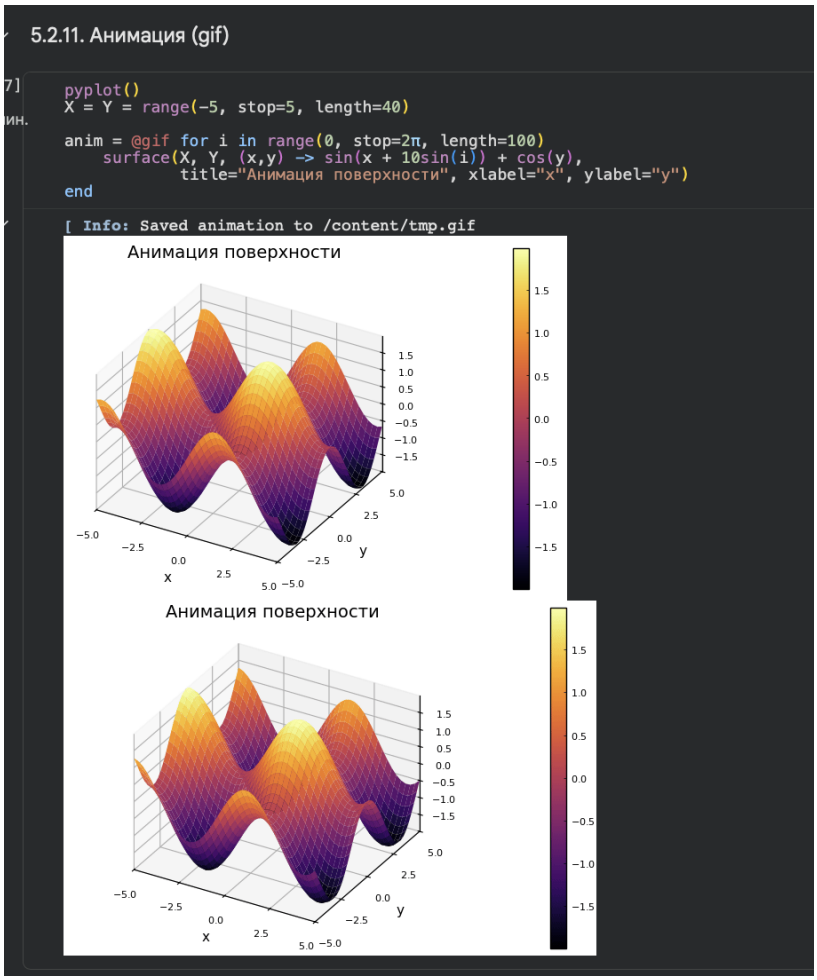


Рисунок 4.18: Анимация поверхности

4.10 5.2.12. Errorbars

Что делаем. Для разных стандартных отклонений `sds` считаем средние значения и строим точки с вертикальными «усами» ошибки $1.96 \cdot \mathrm{std}/\sqrt{n}$ (доверительный интервал 95%).

5.2.12. Errorbars

```
using Plots
gr()

sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]
n = 10
y = [mean(sd .* randn(n)) for sd in sds]
errs = 1.96 .* sds ./ sqrt(n)

plot(y, ylims=(-1, 1), err=errs,
     title="Значения с доверительными интервалами",
     xlabel="номер точки", ylabel="y",
     label="y ± 1.96σ", legend=:topright)
```

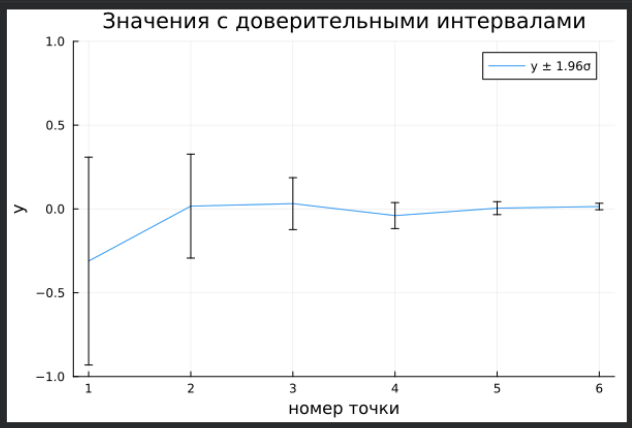


Рисунок 4.19: Errorbars

4.11 5.2.13. Гистограммы (Distributions)

Случайные возрасты. 1000 целых чисел от 15 до 55; гистограмма показывает равномерность.

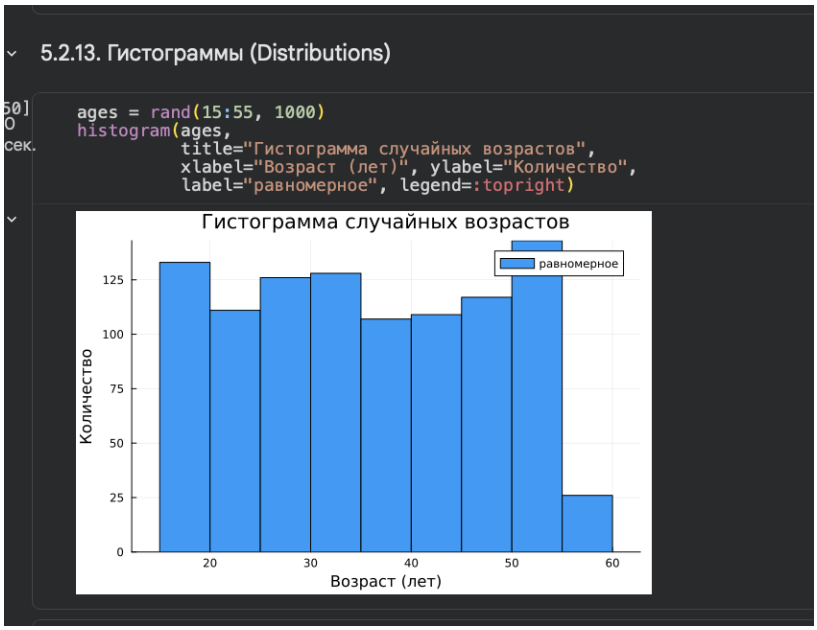


Рисунок 4.20: Гистограмма случайных возрастов

Нормальное распределение. `Normal(35.0, 10.0)`, 1000 значений — колоколообразная гистограмма.

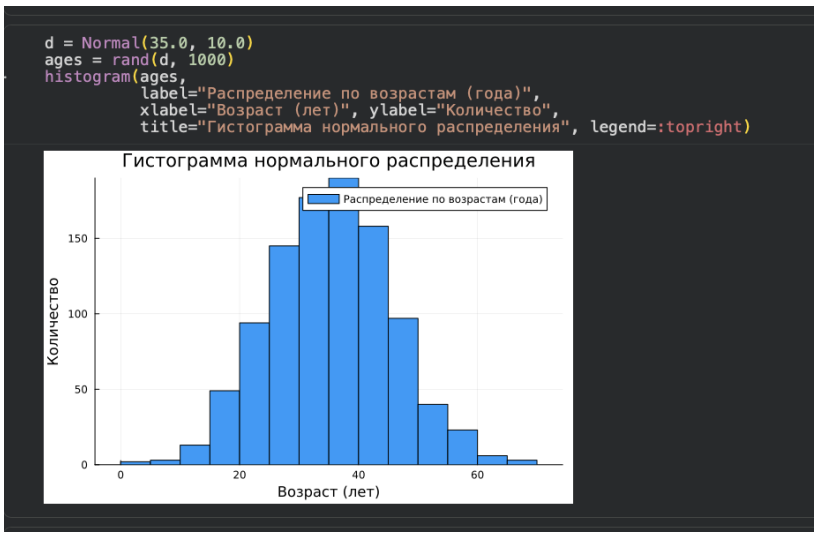


Рисунок 4.21: Гистограмма нормального распределения

Смесь распределений. Три нормальных распределения с разными средними/дисперсиями объединены в один массив; 50 бинов показывают

три «горба».

```
using Plots
gr()

using Distributions
d1 = Normal(10.0, 5.0); d2 = Normal(35.0, 10.0); d3 = Normal(60.0, 5.0)
N = 1000
ages = Float64[]
append!(ages, rand(d1, Int64(ceil(N/2))))
append!(ages, rand(d2, N))
append!(ages, rand(d3, Int64(ceil(N/3))))

histogram(ages, bins=50,
          label="Распределение по возрастам (года)",
          xlabel="Возраст (лет)", ylabel="Количество",
          title="Распределение по возрастам (года)", legend=:topright)
```

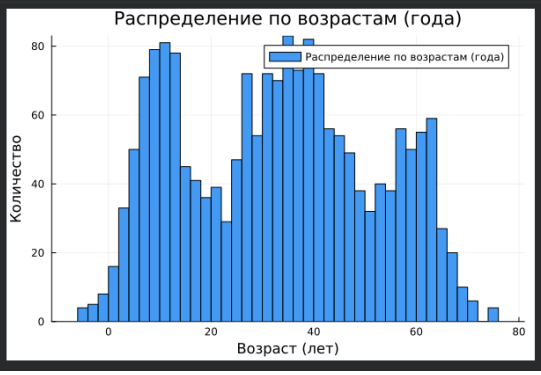


Рисунок 4.22: Смесь распределений

4.12 5.2.14. Подграфики

В столбик. `layout=(4,1)` — четыре ряда графиков.

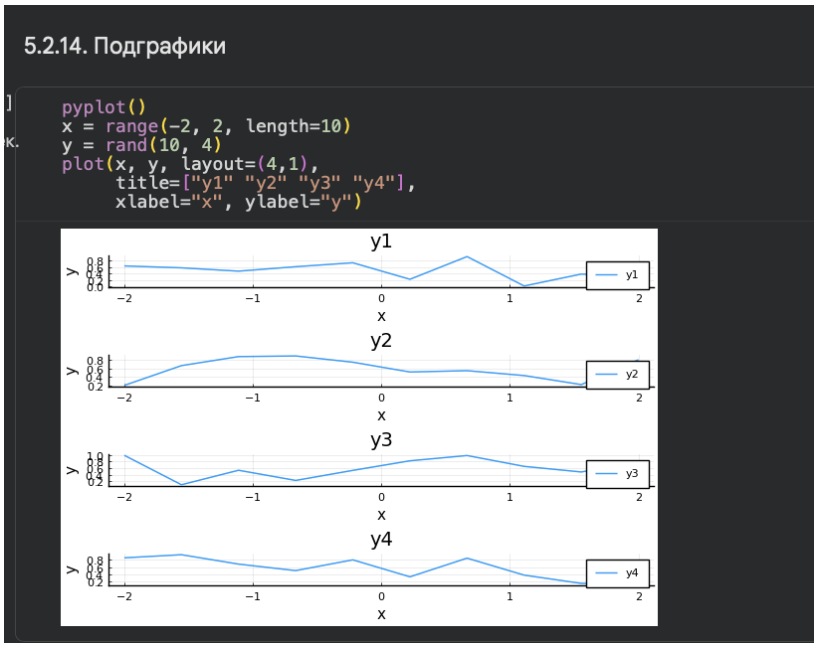


Рисунок 4.23: Подграфики в столбик

Сеткой 2x2. `layout=4` — автоматическая сетка.

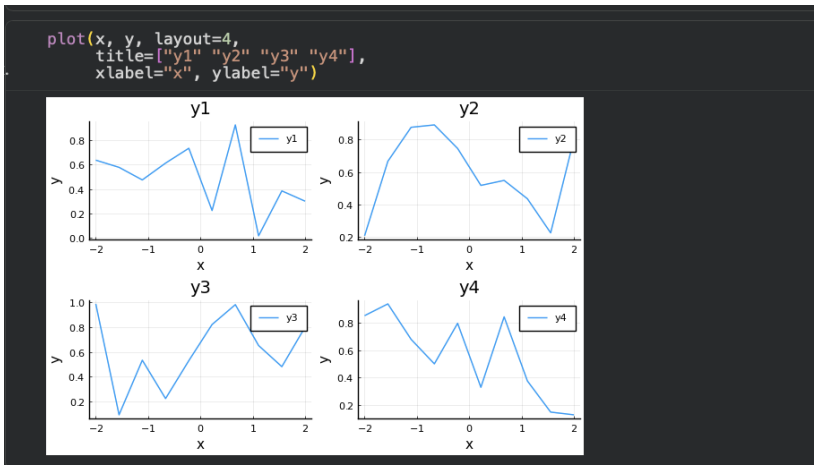


Рисунок 4.24: Подграфики сеткой 2x2

Объединение разных типов. Четыре разных графика (линии, точки, широкие линии, гистограмма) собираются в сетку 2x2 с общим фоном и размером.

```
p1 = plot(x, y, title="Линии", label="y")
p2 = scatter(x, y, title="Точки", label="y")
p3 = plot(x, y[:, 1:2], xlabel="Labelled plot of two columns",
          lw=2, title="Wide lines", label=["y1" "y2"])
p4 = histogram(x, y, title="Гистограммы", label="y")

plot(p1, p2, p3, p4, layout=(2,2), legend=false,
     size=(800, 600), background_color=:ivory)
```

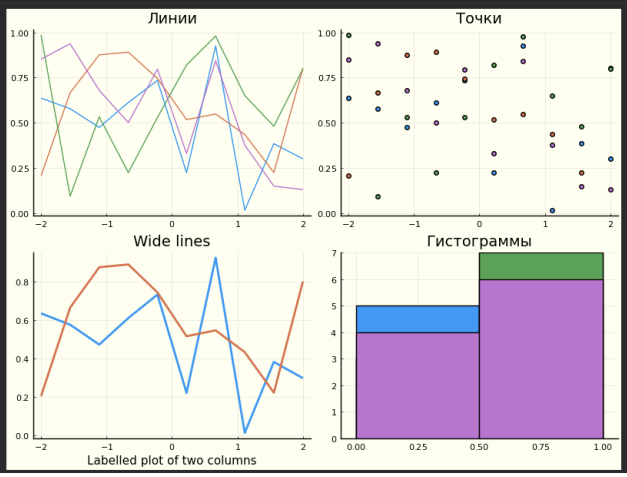


Рисунок 4.25: Объединение разных типов графиков

5 Самостоятельные задания (5.4)

5.1 Задание 1. Все типы графиков sin(x) в одном окне

Что делаем. Для $y = \sin(x)$, $x \in [0, 2\pi]$, строим шесть графиков в сетке 2x3: линия, точки, столбцы (`:bar`), гистограмма, стики (`:sticks`), заливка (`fillrange=0`). Цель — увидеть, как один и тот же набор данных выглядит в разных типах графиков.

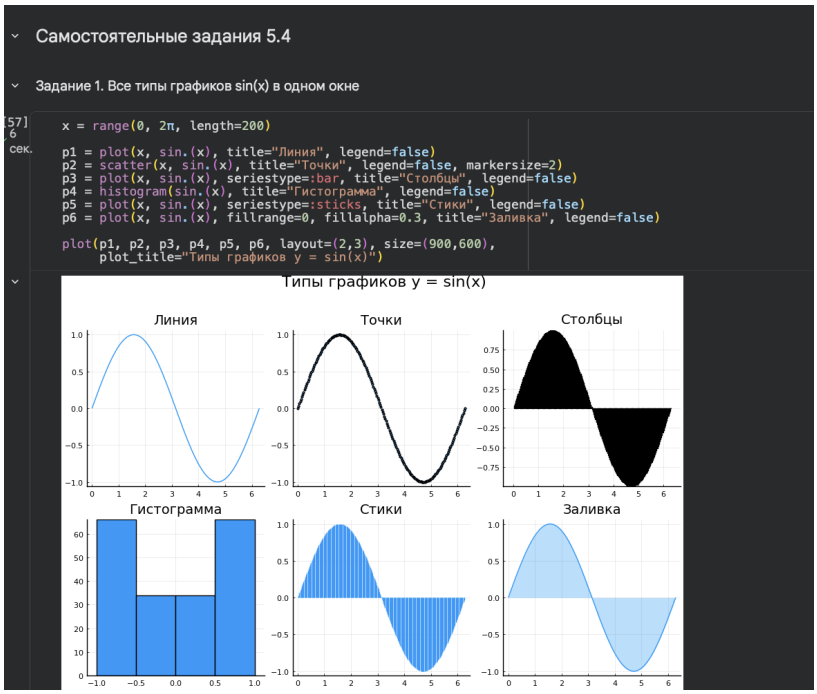


Рисунок 5.1: Все типы графиков sin(x)

5.2 Задание 2. sin(x) со всеми возможными опциями оформления

Что делаем. Настраиваем максимум опций: красный пунктир (`:dash`), толщина 2, легенда, сетка с прозрачностью, фон `:ivory`, рамка (`framestyle=:box`), ограничения осей, свои метки (ticks) по x с шагом $\pi/2$.

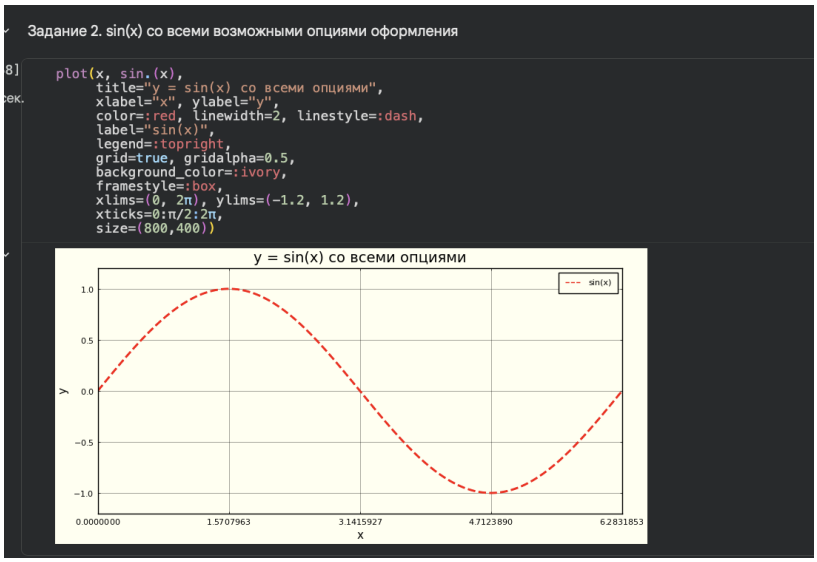


Рисунок 5.2: sin(x) со всеми опциями

5.3 Задание 3. График $y = \pi x^2 \ln(x)$

Что делаем. Строим график на $[0.01, 3]$ (логарифм нуля не существует). Оформление: синяя линия, зелёный фон, рамка, отступы для подписей осей.

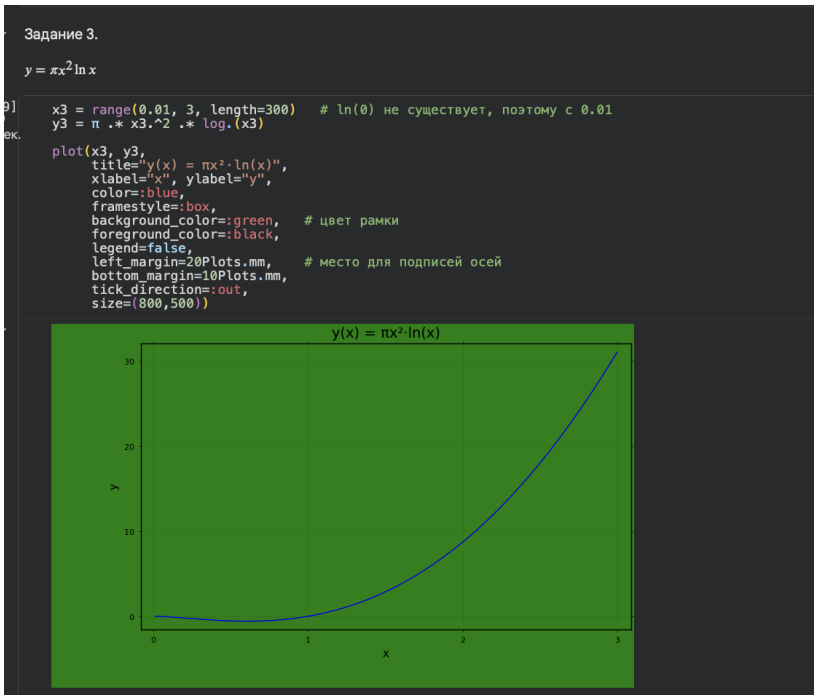


Рисунок 5.3: График y = pi x^2 ln(x)

5.4 Задание 4. Вектор $x = (-2, -1, 0, 1, 2)$, $y = x^3 - 3x$ в 4 панелях

Что делаем. На одном рисунке (сетка 2x2) показываем четыре представления: точки, линию, линию с точками (`:path` + маркеры), сглаженную кривую (на большем числе точек). Сохраняем в `figure_Dagdelen.png`.

Задание 4.

Вектор $x = (-2, -1, 0, 1, 2)$, $y = x^3 - 3x$ в 4-х панелях

```
using Plots
gr()

x4 = [-2, -1, 0, 1, 2]
y4 = x4.^3 .- 3 .* x4

# для плавной кривой берём много точек на том же отрезке
x_smooth = range(-2, 2, length=200)
y_smooth = x_smooth.^3 .- 3 .* x_smooth

q1 = scatter(x4, y4, title="Точки", legend=false)
q2 = plot(x4, y4, title="Линия", legend=false)
q3 = plot(x4, y4, seriestype=:path, marker=:circle, title="Линия + точки", legend=false)
q4 = plot(x_smooth, y_smooth, title="Кривая", legend=false)

plot(q1, q2, q3, q4, layout=(2,2), size=(900,600),
     plot_title="y = x³ − 3x")

savefig("figure_Dagdelen.png")
```

```
"/content/figure_Dagdelen.png"
```

Рисунок 5.4: Задание 4 (4 панели)

Результат сохранения в файл `figure_Dagdelen.png`:

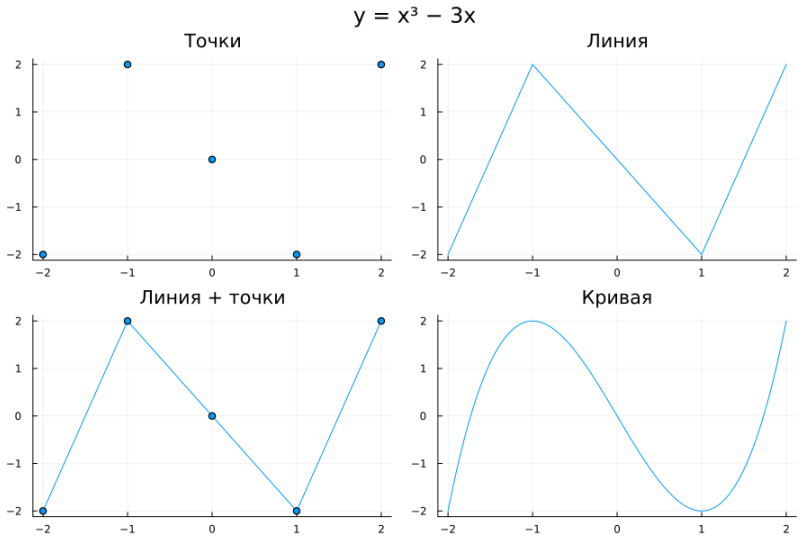


Рисунок 5.5: figure_Dagdelen.png

5.5 Задание 5. Два графика $y_1 = \pi x$, $y_2 = e^x \cos(x)$

Вариант 1. Оба графика на одном рисунке. Недостаток: разные порядки значений — y_1 почти не видна на фоне y_2.

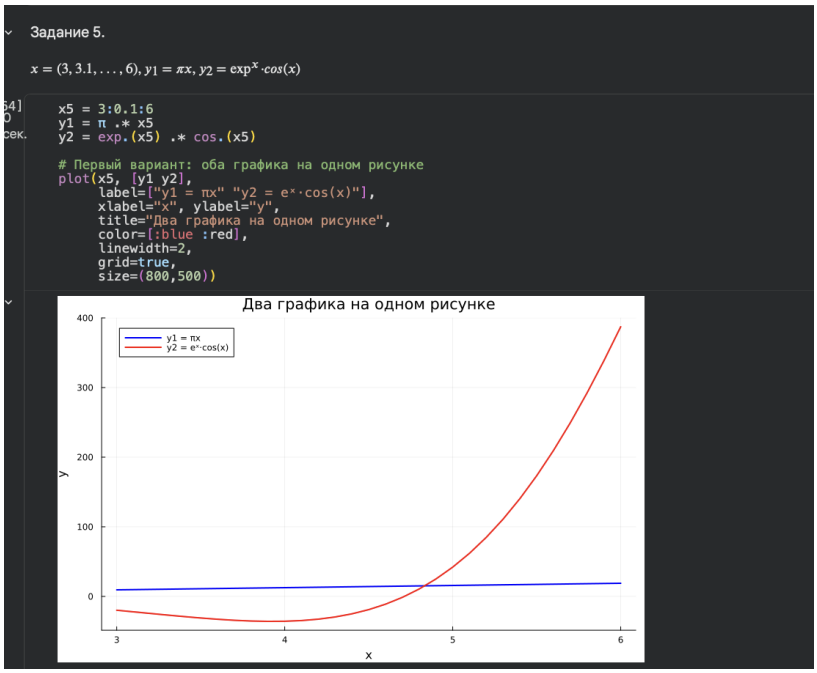


Рисунок 5.6: Задание 5, вариант 1

Вариант 2. Два графика на отдельных панелях (сетка 1x2) — каждая кривая читается.

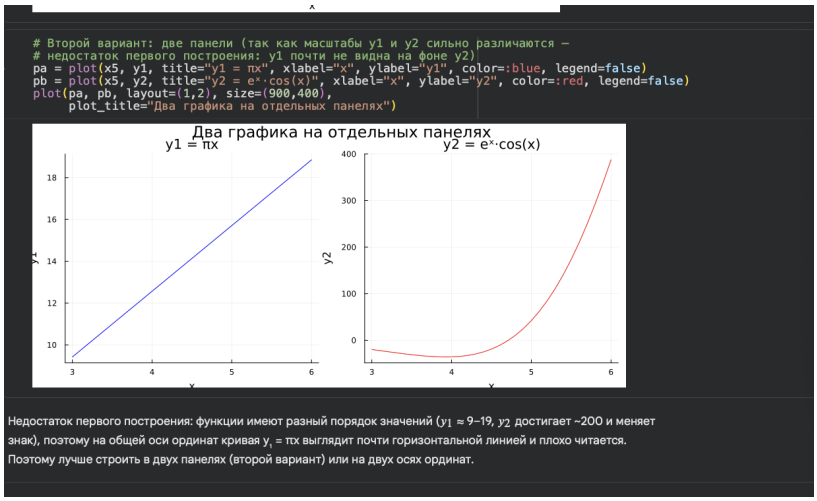


Рисунок 5.7: Задание 5, вариант 2

6 Результаты

В ходе работы получены следующие результаты:

- освоены установка и переключение бэкендов `Plots.jl`;
- построены графики функций, точечные графики (2D и 3D), графики поверхностей, линии уровня, гистограммы, графики с доверительными интервалами, анимация;
- выполнены все самостоятельные задания, скриншоты приложены.

7 Обсуждение результатов

Сравнение бэкендов показало, что `gr()` и `pyplot()` дают статичные изображения, `plotly` — интерактивные, `unicodeplots` — текстовые в консоли. При построении двух функций с разными порядками значений (задание 5) выявлен недостаток совместного размещения на одной оси ординат: одна из кривых становится плохо читаемой. Решение — раздельные панели или две оси ординат.

8 Заключение

Цель работы достигнута: освоен синтаксис Julia для построения графиков. Получены навыки работы с различными типами графиков и опциями оформления. Рекомендуется использовать `plotly` для интерактивных исследований и `gr`/`pyplot` для статичных отчётов.

9 Список литературы

1. *JuliaPlots*. Plots.jl Documentation. — Accessed: 2026-10-30. https://docs.juliaplots.org/.
2. *JuliaStats*. Distributions.jl Documentation. — Accessed: 2026-10-30. https://juliastats.org/Distributions.jl/.
3. *The Julia Language*. Julia Documentation. — Accessed: 2026-10-30. https://docs.julialang.org/.
4. *The Julia Language*. Statistics — Julia Standard Library. — Accessed: 2026-10-30. https://docs.julialang.org/en/v1/stdlib/Statistics/.